`consteval` needs to propagate up

Document #:	P2564R1
Date:	2022-11-08
Project:	Programming Language C++
Audience:	CWG

Reply-to:	Barry “Patch” Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2564R0], many wording changes and added lots of examples.

§ 2 Introduction

[P1240R2] proposes that we should use a monotype, `std::meta::info`, as part of a value-based reflection. One argument here is that lots of operations return ranges of `meta::info` and it would be valuable if we could simply reuse our plethora of existing range algorithms for these use-cases.

Let’s investigate this claim.

We don’t need a working implementation of P1240 to test this, it’s enough to have this very, very loose approximation:

```
namespace std::meta {
    struct info { int value; };

    consteval auto is_invalid(info i) -> bool {
```

```
        // we do not tolerate the cult of even here
        return i.value % 2 == 0;
    }
}
```

And let's pick a simple problem. We start some sequence of… we'll call them types:

```
constexpr std::meta::info types[] = {1, 3, 5};
```

We want to ensure that none of them are invalid. This is a problem for which we have a direct algorithm: `none_of`. Let's try using it:

Attempt	Result

`static_assert(std::ranges::none_of(`
`  types,`
`  std::meta::is_invalid`
`));`
	❌. Ill-formed per 7.5.4.1 [expr.prim.id.general]/4:
	4 A potentially-evaluated id-expression that denotes an immediate function shall appear only
	(4.1) — as a subexpression of an immediate invocation, or
	(4.2) — in an immediate function context.
	Neither of those cases apply here.

`static_assert(std::ranges::none_of(`
`  types,`
`  [](std::meta::info i) {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`));`
	❌. Ill-formed per 7.7 [expr.const]/13:
	13 An expression or conversion is in an immediate function context if it is potentially evaluated and either:
	(13.1) — its innermost enclosing non-block scope is a function parameter scope of an immediate function, or
	(13.2) — its enclosing statement is enclosed ([stmt.pre]) by the compound-statement of a consteval if statement ([stmt.if]).
	An expression or conversion is an immediate invocation if it is a potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate

	invocation shall be a constant expression. `std::meta::is_invalid` is an immediate function, the call to it in the lambda is not in an immediate function context, so it's an immediate invocation. But `std::meta::is_invalid(i)` isn't a constant expression, because of the parameter `i` here.

`static_assert(std::ranges::none_of(`
`  types,`
`  [](std::meta::info i) consteval`
`  {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`));`
	❌. Ill-formed per 7.7 [expr.const]/13 again. This time the lambda itself is fine, but now invoking the lambda from inside of `std::ranges::none_of` is the problem.

`static_assert(std::ranges::none_of(`
`  types,`
`  +[](std::meta::info i)`
`  consteval {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`));`
	❌. Ill-formed per 7.7 [expr.const]/11:
	
	11 A constant expression is either a glvalue core constant expression that refers to an entity that is a permitted result of a constant expression (as defined below), or a prvalue core constant expression whose value satisfies the following constraints:
	
	(11.1) — if the value is an object of class type, each non-static data member of reference type refers to an entity that is a permitted result of a constant expression,
	
	(11.2) — if the value is of pointer type, it contains the address of an object with static storage duration, the address past the end of such an object ([expr.add]), the address of a non-immediate function, or a null pointer value,
	
	(11.3) — if the value is of pointer-to-member-function type, it does not designate an immediate function, and
	
	(11.4) — if the value is an object of class or array type, each subobject

	satisfies these constraints for the value. An entity is a permitted result of a constant expression if it is an object with static storage duration that either is not a temporary object or is a temporary object whose value satisfies the above constraints, or if it is a non-immediate function. Here, explicitly converting the lambda to a function pointer isn't a permitted result because it's an immediate function.

That exhausts the options here. What if instead of trying to directly `static_assert` the result of an algorithm (or, equivalently, use it as an initializer for a constexpr variable or as the condition of an `if constexpr` or as a non-type template argument), we did it inside of a new `consteval` function?

Attempt	Result

`consteval auto all_valid() -> bool {`
`  return std::ranges::none_of(`
`  types,`
`  std::meta::is_invalid`
`  );`
`}`
`static_assert(all_valid());`
	✅. This one is actually valid per the language rules. Except, per the library rules, it's unspecified per 16.4.5.2.1 [namespace.std]/6:
	
	6 Let F denote a standard library function ([global.functions]), a standard library static member function, or an instantiation of a standard library function template. Unless F is designated an addressable function, the behavior of a C++ program is unspecified (possibly ill-formed) if it explicitly or implicitly attempts to form a pointer to F.

`consteval auto all_valid() -> bool {`
`  return std::ranges::none_of(`
`  types,`
`  [](std::meta::info i) {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`  );`
`}`
`static_assert(all_valid());`
	❌. Ill-formed per 7.7 [expr.const]/13 still.

`consteval auto all_valid() -> bool {`
`  return std::ranges::none_of(`
`  types,`
`  [](std::meta::info i)`
`  consteval {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`  );`
`}`
`static_assert(all_valid());`
	❌. Ill-formed per 7.7 [expr.const]/13 still.

`consteval auto all_valid() -> bool {`
`  return std::ranges::none_of(`
`  types,`
`  +[](std::meta::info i)`
`  consteval {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`  );`
`}`
`static_assert(all_valid());`
	✅. Valid. Language and library both.

This leaves a lot to be desired. The only mechanism completely sanctioned by the language and library rules we have today is to write a bespoke consteval function which invokes the algorithm using a non-generic, consteval lambda that just wraps an existing library function and converts it to a function pointer.

Put simply: algorithms are basically unusable with the consteval rules we have.

§ 3 `consteval` is a color

The problem is that `consteval` is a color [what-color]. `consteval` functions and function templates can only be called by other `consteval` functions and function templates.

`std::ranges::none_of`, like all the other algorithms in the standard library, and probably all the other algorithms outside of the standard library, are not `consteval` functions or function templates. Moreover, none of these algorithms are ever going to either become `consteval` or be duplicated in order to gain a `consteval` twin. Which means that none of these algorithms, including `none_of`, can ever invoke any of the facilities proposed by [P1240R2].

§ 3.1 Some code is already multi-colored

Almost.

One of the new C++23 features is `if consteval` [P1938R3], which partially alleviates the `consteval` color problem. That facility allows `consteval` functions to be called by `constexpr` functions in a guarded context, largely in order to solve the same sort of problem that `std::is_constant_evaluated()` [P0595R2] set out to solve except in a way that would now allow you to invoke `consteval` functions.

The way it works is by establishing an immediate function context. This is important because the rule we were constantly running up against was that:

> 13 An immediate invocation shall be a constant expression.

And a call to an immediate function that is in an immediate function context is not an immediate invocation.

In our case here, we're not trying to choose between a compile-time friendly algorithm and a run-time efficient one. We're only trying to write compile-time code. But that's okay, there's no rule that says you need an `else`:

Attempt	Result

`static_assert(std::ranges::none_of(`
`  types,`
`  [](std::meta::info i) {`
`  if consteval {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`  }));`
	✅. Valid.

`static_assert(std::ranges::none_of(`
`  types,`
`  [](std::meta::info i) consteval {`
`  if consteval {`
`  return`
`  std::meta::is_invalid(i);`
`  }`
`  }));`
	❌. Ill-formed per 7.7 [expr.const]/13. Still.

So… this is fine. We can still use `consteval` functions with lambdas, as long as we just wrap all of our lambda bodies in `if consteval` (but definitely also not make them `consteval` - they're only kind of `consteval`).

§ 3.2 Making the library multi-colored

We could avoid putting the burden of sprinkling their code with `if consteval` by pushing all of these extra calls into the library.

For instance, we could make this change to `none_of`:

Today	Tomorrow?

`template <ranges::input_range R, class Pred>`
`constexpr auto ranges::none_of(R&& r, Pred pred) -> bool {`
`  auto first = ranges::begin(r);`
`  auto last = ranges::end(r);`
`  for (; first != last; ++first) {`
`  if (pred(*first)) {`
`  return false;`
`  }`
`  }`
`  return true;`
`}`
	`template <ranges::input_range R, class Pred>`
	`constexpr auto my::none_of(R&& r, Pred pred) -> bool {`
	`  auto first = ranges::begin(r);`
	`  auto last = ranges::end(r);`
	`  for (; first != last; ++first) {`
	`  if consteval {`
	`  if (pred(*first)) {`
	`  return false;`
	`  }`
	`  } else {`
	`  if (pred(*first)) {`
	`  return false;`
	`  }`
	`  }`
	`  }`
	`  return true;`
	`}`

But unfortunately that doesn't actually work. If `pred(*first)` were actually a `consteval` call, then even duplicating the check in both branches of an `if consteval` doesn't help us. The call to `pred(*first)` in the first sub-statement (the case where we are doing constant evluation) is fine, since now we're in an immediate function context, but the call to `pred(*first)` in the second sub-statement (the "runtime" case) is just as problematic as it was without the `if consteval`.

So the attempted solution on the right isn't just ridiculous looking, it also doesn't help. The only library solution here (and I'm using the word solution fairly loosely) is to have one set of algorithms that are `constexpr` and a completely duplicate set of algorithms that are `consteval`.

This has to be solved at the language level.

§ 3.3 Making the language multi-colored

Let's consider the problem in a very local way, going back to the lambda example and presenting it instead as a function (in order to simplify things a bit):

ill-formed	ok

`constexpr auto pred_bad(std::meta::info i) -> bool {`
`  return std::meta::is_invalid(i);`
`}`
	`constexpr auto pred_good(std::meta::info i) -> bool {`
	`  if consteval {`
	`  return std::meta::is_invalid(i);`
	`  }`
	`}`

As mentioned multiple times, `pred_bad` is ill-formed today because it contains a call to an immediate function outside a consteval context and that call isn't a constant expression. That is one way we achieve the goal of the `consteval` functions that they are only invoked during compile time. But `pred_good` is good because that call only appears in an `if consteval` branch (i.e. a consteval context), which makes the call safe.

What's interesting about `pred_good` is that while it's marked `constexpr`, it's actually *only* meaningful during compile time (in this case, it's actually UB at runtime since we just flow off the end of the function). So this isn't really a great solution either. We need to ensure that `pred_good` is only called at compile time.

But we have a way to ensure that: `consteval`.

Put differently, `pred_bad` is today ill-formed, but only because we need to ensure that it's not called at runtime. If we could ensure that, then we calling it during compile time is otherwise totally fine. What if the language just did that ensuring for us? If such `constexpr` functions, that are only ill-formed because of calls to `consteval` functions simply became `consteval`, then we gain the ability to use them at compile time without actually losing anything - we couldn't call them at runtime to begin with.

4 Proposal

This paper proposes avoiding the `consteval` coloring problem (or, at least, mitigating its annoyances) by allowing certain existing `constexpr` functions to implicitly become `consteval` functions when those functions can already only be invoked during compile time anyway.

Specifically, these three rules:

1. If a `constexpr` function contains a call to an immediate function outside of an immediate function context, and that call itself isn't a constant expression, said `constexpr` function implicitly becomes a `consteval` function. This is intended to include lambdas, function template specializations, special member functions, and should cover member initializers as well.

2. If an *expression-id* designates a `consteval` function without it being an immediate call in such a context, it also makes the context implicitly consteval. Such *expression-id*'s are also allowed in contexts that are manifestly constant evaluated.

3. Other manifestly constant evaluated contexts (like *constant-expression* and the condition of a constexpr if statement) are now considered to be immediate function contexts.

With these rule changes, no library changes are necessary, and any way we want to write the original call just works:

Attempt	Proposed

`static_assert(std::ranges::none_of(`
`  types,`
`  [](std::meta::info i) {`
	✅. First, the lambda becomes implicitly `consteval` due to the non-constant call `is_invalid(i)`. This, in turn, makes this

`        return`
`        std::meta::is_invalid(i);`
`    }));`
	instantiation of `ranges::none_of` implicitly `consteval`. And then everything else just works.

`static_assert(std::ranges::none_of(`
`    types,`
`    [](std::meta::info i) consteval`
`        {`
`        return`
`        std::meta::is_invalid(i);`
`    }));`
	✅. Now, the lambda is explicitly `consteval` instead of implicitly `consteval`, which likewise also makes this instantiation of `ranges::none_of` implicitly `consteval`. Everything else just works.

`static_assert(std::ranges::none_of(`
`    types,`
`    std::meta::is_invalid`
`));`
	✅. Still bad based on the library wording, but from the second proposed rule `std::meta::is_invalid` is usable in this context because it is manifestly constant evaluated. `ranges::none_of` does not become `consteval` here, since it does not need to do so.

`static_assert(std::ranges::none_of(`
`    types,`
`    +[](std::meta::info i)`
`        consteval {`
`        return`
`        std::meta::is_invalid(i);`
`    }`
`));`
	✅. Previously, this was ill-formed because the conversion to function pointer needed to be (in of itself) a constant expression, but with the third proposed rule this conversion would now occur in an immediate function context. The “permitted result” rule no longer has to apply, so this is fine. As above, `ranges:none_of` here does not become `consteval`.

`static_assert(std::ranges::none_of(`
`    types,`
`    []{`
`        return`
`        std::meta::is_invalid;`
`    }()`
`));`
	✅. The use of `std::meta::is_invalid` causes the lambda to be `consteval`, through the second rule. And the third rule cases the invocation of the lambda to be in an immediate function context. This produces a function pointer which does not have to be a permitted result of a constant expression, because the invocation no longer needs to be constant expression. In this case too, `ranges::none_of` does not become `consteval`.

§ 4.1 Implementation Experience

This has been implemented in EDG by Daveed Vandevoorde. One interesting example he brings up:

```
consteval int g(int p) { return p; }
template<typename T> constexpr auto f(T) { return g; }
int r = f(1)(2);      // proposed ok
int s = f(1)(2) + r;  // error
```

Today, even the call `f(1)` is ill-formed, because naming `g` isn’t allowed in that context (it is neither a subexpression of an immediate invocation nor in an immediate function context).

Per the proposal, the initialization of `r` becomes valid. `f` implicitly becomes a `consteval` function template due to use of `g`. Because `r` is at namespace scope, we tentatively try to perform constant initialization, which makes the initial parse manifestly constant evaluated. In such a context, `f(1)` does not have to be a constant expression, so the fact that we're returning a pointer to consteval function is okay. The subsequent invocation `g(2)` is fine, and initializes `r` to `2`.

But even with this proposal, the initialization of `s` is ill-formed. The tentative constant initialization fails (because `r` isn't a constant), and in the subsequent dynamic initialization, `f(1)` is now actually an immediate invocation (`f` still becomes implicitly `consteval`, which now must be a constant expression, which now has the rule that its result must be a permitted result, in which context returning a pointer to consteval function is disallowed).

§ 4.2 Wording Circularity

One of the issues with actually wording this proposal is its chicken-and-egg nature. Let's consider the main example again:

```
[](std::meta::info i) {
    return std::meta::is_invalid(i);
}
```

And let's work through both the status quo reasoning and the proposed reasoning.

Current Reasoning	Proposed Reasoning
1. `std::meta::is_invalid` is an *immediate function* because it is declared `consteval` (these are synonyms). 2. The lambda's call operator is not an immediate function, because it is not declared `consteval`. 3. The expression `std::meta::is_invalid(i)` is not in an *immediate function context* because of (2) and also it is not in an `if consteval`. 4. The combination of (1) and (3) make the expression `std::meta::is_invalid(i)` an *immediate invocation*, which is required to be a constant expression. 5. That expression isn't a constant expression because of the use of the	1. `std::meta::is_invalid` is an *immediate function* because it is declared `consteval`. 2. The lambda's call operator is not an immediate function, because it is not declared `consteval`. 3. The expression `std::meta::is_invalid(i)` is not in an *immediate function context* because of (2) and also it is not in an `if consteval` and also it is not manifestly-constant-evaluated. 4. The combination of (1) and (3) make the expression `std::meta::is_invalid(i)` an *immediate-escalating expression*. 5. The lambda's call operator is declared neither `constexpr` nor `consteval`, which makes it an *immediate-escalating function*.

function parameter, `i`, so this is ill-formed.	6. The combination of (4) and (5) cause the lambda's call operator to become an *immediate function*. 7. … which means that now the expression `std::meta::is_invalid(i)` is in an immediate function context. 8. Which now means that the expression `std::meta::is_invalid(i)` is no longer an immediate invocation.

Essentially, we have a flow of reasoning that starts with a function *not* being an immediate function and, because of that, becoming an immediate function. This is, admittedly, confusing. But I think it does make sense, and Daveed had less trouble implementing this than we had even attempting to try to reason about wording it.

§ 5 Wording

Extend 7.7 [expr.const]/13:

13 An expression or conversion is in an *immediate function context* if it is potentially evaluated and either:

(13.1) — its innermost enclosing non-block scope is a function parameter scope of an immediate function, ~~or~~

(13.2) — it is a subexpression of a manifestly constant-evaluated expression or conversion, or

(13.3) — its enclosing statement is enclosed ([stmt.pre]) by the *compound-statement* of a consteval if statement ([stmt.if]).

An expression or conversion is an *immediate invocation* if it is a potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. ~~An immediate invocation shall be a constant expression.~~

13a An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and it is either

(13a.1) — a potentially-evaluated *id-expression* that denotes an immediate function that is not a subexpression of an immediate invocation, or

(13a.2) — an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.

13b An *immediate-escalating* function is:

(13b.1) — the call operator of a lambda that is not declared with the consteval specifier,

(13b.2) — a defaulted special member function that is not declared with the consteval specifier, or

(13b.3) — a function that results from the instantiation of a templated entity defined with the constexpr specifier.

An immediate-escalating expression shall appear only in an immediate-escalating function.

13c An *immediate function* is a function or constructor that is:

(13c.1) — declared with the `consteval` specifier, or

(13c.2) — an immediate-escalating function F whose function body contains an immediate-escalating expression E such that E's innermost enclosing non-block scope is F's function parameter scope.

[*Example*:

```
consteval int id(int i) { return i; }
constexpr char id(char c) { return c; }

template <typename T>
constexpr int f(T t) {
    return t + id(t);
}

auto a = &f<char>; // ok, f<char> is not an immediate function
auto b = &f<int>;  // error: f<int> is an immediate function

static_assert(f(3) == 6); // ok

template <typename T>
constexpr int g(T t) {    // g<int> is not an immediate function
    return t + id(42);    // because id(42) is already a constant
}

template <typename T, typename F>
constexpr bool is_not(T t, F f) {
    return not f(t);
}

consteval bool is_even(int i) { return i % 2 == 0; }

static_assert(is_not(5, is_even)); // ok

int x = 0;

template <typename T>
constexpr T h(T t = id(x)) { // h<int> is not an immediate function
    return t;
}

template <typename T>
constexpr T hh() {           // hh<int> is an immediate function
    return h<T>();
}
```

```
int i = hh<int>(); // ill-formed: hh<int>() is an immediate-escalating
        expression
                   // outside of an immediate-escalating function

-end example]
```

Remove 7.5.4.1 [expr.prim.id.general]/4 (it is handled above in the definition of immediate-escalating).

4 ~~A potentially-evaluated id-expression that denotes an immediate function shall appear only~~

(4.1) — ~~as a subexpression of an immediate invocation, or~~

(4.2) — ~~in an immediate function context.~~

And removing the current definition of *immediate function* from 9.2.6 [dcl.constexpr]/2, since it's now defined (recursively) above.

2 A `constexpr` or `consteval` specifier used in the declaration of a function declares that function to be a *constexpr function*. [*Note*: A function or constructor declared with the consteval specifier is ~~called~~ an immediate function ([expr.const]) *-end note*]. A destructor, an allocation function, or a deallocation function shall not be declared with the consteval specifier.

§ 6 Acknowledgments

Thanks to Daveed Vandevoorde and Tim Song for discussions around this issue and Daveed for implementing it.

§ 7 References

[P0595R2] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018-11-09. std::is_constant_evaluated.
https://wg21.link/p0595r2

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.
https://wg21.link/p1240r2

[P1938R3] Barry Revzin, Daveed Vandevoorde, Richard Smith, Andrew Sutton. 2021-03-22. if consteval.
https://wg21.link/p1938r3

[P2564R0] Barry Revzin. 2022-03-15. consteval needs to propagate up.
https://wg21.link/p2564r0

[what-color] Bob Nystrom. 2015-02-01. What Color is Your Function?
https://journal.stuffwithstuff.com/2015/02/01/what-color-is-your-function/